# INTEGRATION OF AN AI AGENT CHATBOT INTO AN EXISTING PROFESSIONAL SERVICES AUTOMATION APPLICATION

Master's thesis

Lappeenranta–Lahti University of Technology LUT

Master's Program in Software Engineering and Digital Transformation

2026

Joonatan Kallio

Examiners: Full Professor Jussi Kasurinen (D.Sc.)

University Lecturer Erno Vanhala (D.Sc.)

Leevi Kämäräinen (M.Sc.)

ABSTRACT

Lappeenranta–Lahti University of Technology LUT

LUT School of Engineering Science

Software Engineering

Joonatan Kallio

**Integration of an AI agent chatbot into an existing Professional Services Automation application**

Master's thesis

2026

59 pages, 10 figures, 1 table and 1 appendix

Examiners: Full professor Jussi Kasurinen (D.Sc.), University lecturer Erno Vanhala (D.Sc.), Leevi Kämäräinen (M.Sc.)

Keywords: Artificial intelligence, user experience, user interface, usability

Artificial intelligence has risen in popularity in recent years. This called for an investigation into user experience and usability within artificial intelligence agent chatbots. The case company had started planning for a new AI agent feature to be integrated into the case application. The case application is Professional Services Automation (PSA) software, and it did not include any major agentic AI features prior to this thesis. This thesis used design science research method to answer the research questions. In the design science research method, an artifact is created and then evaluated. The artifact was an AI agent chatbot. During development, the best practices for integrating an AI feature into an existing application were noted. The artifact was tested internally and then feedback was collected via a survey regarding the usability of the artifact. The survey results indicated that the usability of the created artifact was average. All three artifact objectives, which were set, were reached. This thesis suggests future research to focus on the usability of AI tools with chatbot-specific questionnaire. The survey sample size and the number of responses were small, and the survey participants were employees of the case company, so real customers and users of the case application were not surveyed. This limits the findings and therefore the findings cannot be generalized to a broader number of applications or systems.

TIIVISTELMÄ

Lappeenrannan–Lahden teknillinen yliopisto LUT

LUT Teknis-luonnontieteellinen

Tietotekniikka

Joonatan Kallio

**Tekoälyagentti-keskustelubotin integrointi olemassa olevaan asiantuntijayrityksen toiminnanohjausjärjestelmään**

Tietotekniikan diplomityö

2026

59 sivua, 10 kuvaa, 1 taulukko ja 1 liite

Tarkastajat: Professori Jussi Kasurinen (TkT), Yliopisto-opettaja Erno Vanhala (TkT) ja Diplomi-insinööri Leevi Kämäräinen

Avainsanat: Tekoäly, käyttäjäkokemus, käyttöliittymä, käytettävyys

Tekoälyn suosio on kasvanut viimeisten vuosien aikana. Tämä johti tutkimukseen käyttäjäkokemuksesta sekä käytettävyydestä tekoälyagenttien keskusteluboteissa. Case-yritys oli jo aloittanut suunnitelmat uuden tekoälyagenttiominaisuuden integroimisesta case-sovellukseen. Case-sovellus on asiantuntijayrityksen toiminnanohjausohjelmisto, jossa ei aiemmin ole ollut suurempia agentti-tekoälyominaisuuksia. Tässä diplomityössä käytettiin suunnittelututkimusmenetelmää tutkimuskysymyksiin vastaamiseksi. Suunnittelututkimusmenetelmässä luodaan artefakti ja arvioidaan sitä. Artefakti oli tekoälyagenttia hyödyntävä keskustelubotti. Kehitysvaiheen aikana parhaimmat käytänteet tekoälyagentin integroimisesta olemassa olevaan sovellukseen kirjoitettiin ylös. Artefakti testattiin case-yrityksessä sisäisesti ja palaute artefaktin käytettävyydestä kerättiin kyselytutkimuksen avulla. Kyselytutkimuksen tulokset osoittavat luodun artefaktin käytettävyyden olevan keskiluokkaa. Kaikki kolme artefaktille asetettua tavoitetta saavutettiin. Tämä diplomityö ehdottaa tulevaisuuden tutkimusten keskittyvän etenkin tekoälyagentti-keskustelubottien käytettävyyden tutkimiseen käyttäen keskusteluboteille tarkoitettuja kyselyitä. Kyselyn otoskoko sekä vastausten määrä oli pieni. Kyselyyn osallistuneet vastaajat olivat case-yrityksen työntekijöitä, eli eivät case-sovelluksen oikeita asiakkaita tai käyttäjiä. Tämä rajoittaa diplomityön löydöksiä eikä niitä voida yleistää muihin sovelluksiin tai järjestelmiin.

ACKNOWLEDGEMENTS

I want to thank my family and friends for supporting me throughout the entire thesis process and motivating me to finish my studies. I also want to thank Erno Vanhala, my supervisor, who gave me valuable feedback and guidance during the writing process.

ABBREVIATIONS

| | |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| BUS-11 | chatBot Usability Scale |
| CRM | Customer Relationship Management |
| CUQ | Chatbot Usability Questionnaire |
| DOM | Document Object Model |
| DS | Design System |
| DSRM | Design Science Research Methodology |
| GDPR | General Data Protection Regulation |
| GUID | Globally Unique Identifier |
| HCI | Human-Computer Interaction |
| HTML | HyperText Markup Language |
| HTTP | Hypertext Transfer Protocol |
| HTTPS | Hypertext Transfer Protocol Secure |
| IS | Information Systems |
| IT | Information Technology |
| JS | JavaScript |
| JSON | JavaScript Object Notation |
| JSX | JavaScript XML |
| LLM | Large Language Model |
| NLP | Natural Language Processing |
| NPM | Node Package Manager |
| PSA | Professional Services Automation |

| | |
|---|---|
| SPA | Single-Page Application |
| SSE | Server-Sent Events |
| SUS | System Usability Scale |
| UI | User Interface |
| URL | Uniform Resource Locator |
| UX | User Experience |
| XHR | XMLHttpRequest |
| XML | Extensible Markup Language |

**Table of contents**

Abstract

Acknowledgements

Symbols and abbreviations

Declarations

1 Introduction ..... 10
1.1 Objectives and delimitations ..... 11
2 Literature review ..... 12
2.1 Usability and user experience ..... 12
2.2 Human-Computer Interaction and AI ..... 13
2.3 Chatbots ..... 14
2.4 Frontend architecture and development ..... 16
2.4.1 Flux architecture ..... 17
2.5 Frontend frameworks ..... 17
2.5.1 React ..... 19
2.6 Server-Sent Events ..... 21
3 Research method ..... 23
3.1 Identifying and defining the problem ..... 24
3.2 Defining the objectives of a solution ..... 25
4 Developing the artifact ..... 26
4.1 Development tools ..... 26
4.2 Initial architecture development ..... 26
4.3 Creating the service ..... 29
4.4 Creating the store and actions ..... 30
4.5 Developing the initial UI ..... 31
4.6 Improving the user experience ..... 33
4.7 Refactoring the services ..... 34
4.8 Transparency of the AI agent ..... 35
4.9 Creating the work hour actions ..... 36
4.10 Additional features ..... 36

5 Demonstration ..... 38
6 Evaluation ..... 43
6.1 Results ..... 45
7 Discussion ..... 47
7.1 Survey results ..... 47
7.2 AI agent ..... 49
7.3 Research questions ..... 50
7.4 Realized research risks and limits ..... 51
8 Conclusions ..... 52
References ..... 53

Appendices

Appendix 1. Summary of the survey results

DECLARATIONS

**AI Usage**

The author of the thesis, Joonatan Kallio, used the following AI-tools during the preparation of the thesis:

1. Cursor
    1. Purpose of this tool: Assisted the author with the development of the artifact.
    2. Explanation of the use of the tool: The tool assisted the author during the development phase of the artifact. All the code suggested by the tool was reviewed and verified by the author before using them.
2. Claude Code
    1. Purpose of this tool: Assisted the author with the development of the artifact.
    2. Explanation of the use of the tool: The tool assisted the author during the development phase of the artifact. All the code suggested by the tool was reviewed and verified by the author before using them.
3. Google Gemini
    1. Purpose of this tool: Grammar checking and explaining complex topics during the writing of this thesis.
    2. Explanation of the use of the tool: The tool assisted in checking grammar from the author's sentences and helped in explaining complex topics to help the author understand them better. All content was reviewed and verified by the author.

**Responsibility**

The author, Joonatan Kallio, takes full responsibility for the content of this thesis and has reviewed and edited the content generated by the possible use of AI tools.

# 1 Introduction

The popularity of artificial intelligence (AI) has been growing in recent years. It is integrating itself into people's daily lives and appears to be a major next step in technology. One key takeaway in the AI Index Report done by Stanford University is that AI is used more and in concrete use cases, not just for research purposes [1]. With the public opinion of AI and its beneficiality slowly rising from 52% in 2022 to 55% in 2024 [1], it is starting to get utilized in multiple fields to improve time-consuming tasks and to make people's lives easier. These use cases can vary from checking the grammar of a given document [2] to generating tests and prioritizing tests for a software project by utilizing AI [3]. It is therefore important to start researching the utilization of AI in people's daily lives and how it could be used to improve the existing software solutions. These software solutions are used daily by many people, therefore the improvements in user experience are welcome.

The utilization of AI can also be seen in applications that use a chatbot where the usual human support is replaced with an artificial intelligence agent that answers users' questions [4]. These kinds of applications of artificial intelligence can improve the user experience by allowing faster responses than a human support agent would, with a possibility to tailor the responses based on the user's data and previous questions.

The case software in this thesis is a Professional Services Automation (PSA) tool, which is used daily by professionals to track their work hours, manage projects, customers and invoicing. Given the new possible improvements to user experience with artificial intelligence, the case company started to design a completely new AI agent chatbot feature, which allows users to perform daily and repetitive tasks via the chat view. The new AI feature could solve the problem of users having to perform these repetitive tasks manually and often, improving the user experience. The feature would utilize agentic AI, which is capable of doing multiple different actions by using machine learning models that have the capability to even mimic the human decision-making [5]. Agentic AI uses Large Language Models (LLMs) in the background to allow the users to use human languages to interact with the AI [5].

Prior research about AI and especially agentic AI has focused on applications of agentic AI. A comprehensive study found that agentic AI can improve human productivity in

knowledge-intensive tasks [6], which indicates that there is a possibility to improve user experience with AI in PSA solutions. There has not been any research done on implementing agentic AI into PSA software and what kinds of user interface elements used together with AI are beneficial in these types of software.

## 1.1 Objectives and delimitations

This thesis aims to build an agentic artificial intelligence feature and find what kinds of user interface patterns are preferred by the case software's users and what are the most usable user interface design choices in AI agents. This paper also aims to explore the good and bad practices of integrating AI agents into existing PSA solutions, which have existing codebases and coding practices.

This thesis will include a testing phase where the case application team's employees can test out the feature and then answer a survey. The collected data from the survey will be used to answer the research questions and the feature will be improved based on the data. The testing phase will only be conducted internally, and this is due to the time limitations of the thesis work.

The research questions are:

1. What are the technical best practices of integrating artificial intelligence tools into existing software architectures?
2. How do different user interface patterns for agentic artificial intelligence affect the usability and error rates in professional services automation software?
3. Which user interface patterns provide the highest usability for users using an artificial intelligence conversation tool in professional services automation software?

The thesis is done for a case company, which means that the feature will be built together with designers and backend developers from the company. This will limit the scope of this thesis to focus only on the frontend development and analysis of the user interface and user experience.

# 2 Literature review

The literature review analyses the existing research on user interface as well as from agentic artificial intelligence perspective and then highlights the key findings, problems and questions that were discovered during the research. The literature review will focus on the user interface and user experience findings in agentic AI and the technical practices of implementing a new feature into existing software.

## 2.1 Usability and user experience

A lot of usability and user experience research have been conducted. In a study done by Pellas [7], the author studied the user experience of AI chatbots in Philology. The author analysed 26 different undergraduate students and their use of one of the three different agentic AI chatbots, which were the following popular AI tools ChatGPT, Gemini and DeepSeek. By analysing and collecting feedback using surveys, usability scales, and interviews, the author evaluated the usability of the popular AI systems in learning context. The findings were that all three AI systems were usable, with System Usability Scale (SUS) ranging from 70.2 to 78.6. The SUS consists of 10 questions and the maximum score from the survey is 100 and lowest is 0 [8]. In System Usability Scale, the scoring can be compared to university grading scale, where the score of 90-100 is an A, 80-89 is B, so the score that the chatbots received would be C grade, which is considered good [9]. Based on the study's results, DeepSeek was rated with the highest level of usability amongst the participants. The author argued that it could be due to its conversation flow and responsive feedback that the participants of the study had highlighted during the survey. The author also suggested that the overall usability in these three surveyed AI agents is high, with the participants reporting high ease-of-use with all three systems. The study claims that these findings suggest that the conversational agent paradigms generally lead to intuitive interactions for users.

Even though there are studies done with the system usability scale, there is a need for new tools that can assess the AI chatbot user interfaces and experience with reliability. New study was done to examine one of the existing tools, the chatBot Usability Scale (BUS-11), which was created to evaluate customers' or users' relationship with chatbots and the chatbots'

usability. The research mentions that chatbots are a way to convert the traditional point and click user interfaces to be handled with natural language from a chat view. The study points out that the usual difference between the traditional interfaces and chatbots is that chatbots aim to drive the users towards their goal through the natural language. The study focuses mostly on customer relationship management (CRM) chatbots to test the BUS-11 reliability. The research paper measured participants' interactions with random chatbots selected from a pool of 44 chatbots and then analysed the collected data. They concluded that BUS-11 is a robust tool for assessing the usability of chatbots especially within the CRM space. [10]

Another chatbot usability scale is introduced by a conference paper done by Holmes *et al.* [11] which introduces the Chatbot Usability Questionnaire (CUQ). The conference paper chose three different chatbots which were then rated using the CUQ and then the answers were analysed. It also mentioned that the CUQ could be a contender to the SUS which the conference paper deems too outdated to be used for chatbots. The studies show that the SUS scaling might not be enough for ranking the new chatbot user interfaces and there is clearly a need for new methods that consider more than just the traditional 10 questions from SUS.

## 2.2 Human-Computer Interaction and AI

Human-Computer Interaction (HCI) is the field of study between humans and computers and how these two interact with each other. It focuses on improving the usability of systems that humans use, like software, even though traditionally it was between the physical input and output devices and humans. Today, HCI is about development of user-friendly interfaces in websites, games, software and office equipment. [12]

A conference paper done by Kaye *et al.* [13] that discussed how artificial intelligence impacts HCI, the authors discussed that the HCI field is transforming due to the new AI tools. The authors mention that productivity in areas where knowledge is the key is mentioned to be one of the strengths of LLM-based applications. The authors state that most AI applications do not currently give any information on if they are hallucinating, which is a huge problem for the user since they might not notice it either.

## 2.3 Chatbots

Chatbots are conversational artificial intelligence systems, usually called chatbots or AI agents and they are conversational systems made to let AI engage in a conversation with humans by helping them find information about a specific topic [14], [15], [16], [17]. In a research paper done by Nguyen *et al.* [15] the authors compared the traditional menu-based interfaces to the new chatbot interfaces where the users do actions with natural language instead of buttons or other interactable elements. The study highlights that the recent advancements in AI have increased the adoption of AI chatbots to a broader view. Nguyen *et al.* define chatbot as "an AI application that uses Natural Language Processing (NLP) to understand and enable conversation between a human and a machine" [15, p. 2]. According to the study, the chatbots can accomplish and help users in many daily tasks like checking the weather to automating enterprise IT help desk tasks. There has been multiple studies of using AI chatbots for language learning which have concluded that the chatbots can also be used efficiently in learning tasks [16], [18]. One of these chatbot assisted learning tasks was found to be assisting in learning speaking and listening skills [18]. Chatbots could also correct the student's spelling errors [16]. These findings means that the chatbots are not tied only to the conversational dialogue. One of the hypotheses from the paper done by Nguyen *et al.* [15] was that the chatbots will lower the user's cognitive load compared to the traditional menu-based interfaces that are used in traditional software. Another hypothesis set by Nguyen *et al.* was that the chatbot and its interface will lead to lower sense of agency in the users compared to the menu-based interfaces. The cognitive load was studied in the educational context by Srinivasaprasad *et al.* [19] and the authors studied the cognitive load of conversational AI agents on the students. Nguyen *et al.* [15] conducted an experiment where participants would use a chatbot and menu-based interfaces to make a hotel reservation. The findings suggest that chatbots lessen the user's control over the system compared to menu-based system. They noticed that the first time a user was using the chatbot, the users did not give the chatbot the information that could be used to finish the tasks successfully, which slowed down the interaction. This is because the users must know the information that they need to do the specific task and not recognise the information from a given list of menu items. This increases the user's cognitive load. The research suggests that the implementation of chatbots is not a silver bullet that solves all the problems with traditional menu-based interfaces and they could even increase the cognitive load, instead of

decreasing it. The researchers give suggestions on how to reduce the cognitive load, like using both chatbot and menu-based interfaces to give the user for example suggestions in the chat. The study done by Srinivasaprasad *et al.* [19] found out that the chatbots might lower the overall cognitive load of the user, but the agent responses can also increase it, depending on the way the agent answers. They suggest that the agent responses must be concise and not vague to prevent the cognitive load from increasing.

In another survey article Chaves and Gerosa [20] discusses the human-chatbot interaction design and how involving the social characteristics in the chatbots is a useful tool. At the start of the article, they define chatbots as "computer programs that interact with users in natural language" [20, p. 1] and after that, the paper opens the history of chatbots which dates to 1950. The paper claims that even though the chatbots are changing the interaction patterns between users and chatbots, they still fall short of user expectations. The study used a literature survey as the research method and analysed 56 different studies about chatbots. With the data from the different studies, the researchers derived 11 social characteristics and constructed a table that grouped them in three different categories: conversational intelligence, social intelligence and personification. The researchers then explained what these groups meant and according to the study, conversational intelligence means the chatbot's ability to take a part in the conversation with the user and to have the means to achieve the conversation goals. Social intelligence focuses on the social habits of the chatbot, and the personification focuses on the personality that the chatbot uses. The findings that were grouped into these three categories, were also divided into benefits, challenges and strategies.

The conversational intelligence characteristics which were derived from the studies, analysed proactivity, conscientiousness and communicability. Proactivity is about the chatbots' ability to introduce new information and act individually without the user needing to prompt every step. Conscientiousness is about delivering the messages as coherent information and communicability is the chatbots' ability to communicate to the user what its capabilities are. From the social intelligence group there were six characteristics from the total of eleven characteristics which were described in the study done by Chaves *et al.* as damage control, thoroughness, manners, moral agency, emotional intelligence and personalization [20]. The authors say that damage control characteristic contained the chatbot's ability to handle the prompts which were trying to be negative or tried to abuse the

system. Thoroughness is said to be the ability which increases the human-likeness, this is also seen in manners which are categorized in the social intelligence group. Moral agency is there to have the moral compass and to prevent the chatbot from communicating something immoral which could upset the users. Personalization is to make the chatbot more human-like and to adapt to the user's previous prompts to the chatbot. And in the personification category the researchers found two characteristics which are identity and personality. Identity is related to for example giving the chatbot a name and personality is used to increase the believability of the chatbot via language choices. These are all major characteristics and benefits and challenges for the chatbots. The researchers explain that the study had some limitations as they excluded speech-based conversational agents from it. The research suggests that the future research should focus on the degree of influence these benefits provide to the user and users' satisfaction. The future work is also suggested to expand the list of characteristics of a chatbot. A study by Gnewuch *et al.* [17] studied the human-chatbot interactions but instead of focusing on the social characteristics, the study focused on the response time and also analysed the other aspects of human-chatbot interactions. The study found that more experienced AI chatbot users could prefer less humanized chatbot agents. This is because based on the study, the experienced users focused more on the utility compared to the less experienced users, especially when response time was considered. The response time could make it or break it between the novice and experienced users, who appreciated the instant responses instead of human-likeness with the slower responses.

These findings from the papers indicate that the success of human-chatbot interaction depends on multiple factors such as the overall user experience and the social characteristics used in the chatbot as well as the user's knowledge and previous experience of AI agents.

## 2.4 Frontend architecture and development

There are many frontend architecture patterns to choose from, but this thesis only focuses on one of them. Flux architecture is extensively analysed in this thesis due to the case application's architecture being a custom-built Flux based architecture. Flux is mentioned to be a set of architectural patterns, that can be applied in JavaScript (JS) applications. It is built on top of unidirectional data flow, so the data only flows to one way. It is used for managing the data and the state of a frontend application. [21]

### 2.4.1 Flux architecture

Based on the book written by Adam Boduch [21] the Flux architecture has three distinct and individual components which are actions, dispatcher and store. These all have their own specific job within the architecture. The actions are the verbs and the capabilities of the application. For example, whenever a user clicks a button that does something in the application, there is an action for that. The actions contain a payload that can be the data or something else that is needed for the action to be completed.

Dispatcher is the main orchestrator for the Flux architecture according to the book. It handles the distribution of those previously mentioned actions. It will receive the actions and dispatch them to the correct stores that are listening for specific actions. The book talks about the dispatcher with the term “Broker”, which is also correct and it can be seen as that. There is only one dispatcher in the Flux architecture.

The book explains that stores are the place where the state is kept and maintained. Stores handle the state management, so they include the current data and state. These can be changed through the store via the mentioned actions. The store is the only component to make the decision to mutate the state. The book mentions that Flux architecture also unofficially includes a view component. It writes that technically it isn’t a part of Flux, but the architecture affects the views so it could be part of Flux at one point.

## 2.5 Frontend frameworks

During the early days of web development, it was done primarily with HTML and adding JavaScript to the tech stack to get more out of the websites. The earliest sign of a web framework was jQuery in 2006. Fast forward to the present day, there are multiple popular and relevant JavaScript frameworks like React, Angular and Vue. All of these were developed for problems that the previous and existing frameworks couldn’t solve or to solve the problems with another type of solution. React for example was developed by Facebook, first for their own use within the Facebook application but it was later made open source for everyone to use. Angular was released and developed by Google. Vue is the newest of these three and it was published in 2014 by Evan You. [22]

There are multiple surveys that developers can fill out and one of the most popular surveys is the Stack Overflow Developer survey, that is created annually. It compiles data from all parts of software engineering. The following figure is the most popular web frameworks and technologies from the year 2025. The question was “Which web frameworks and web technologies have you done extensive development work in over the past year and which do you want to work with in over the next year?”. There were 23678 answers to this particular question and the respondents varied from professionals to people who are still learning to program. [23]

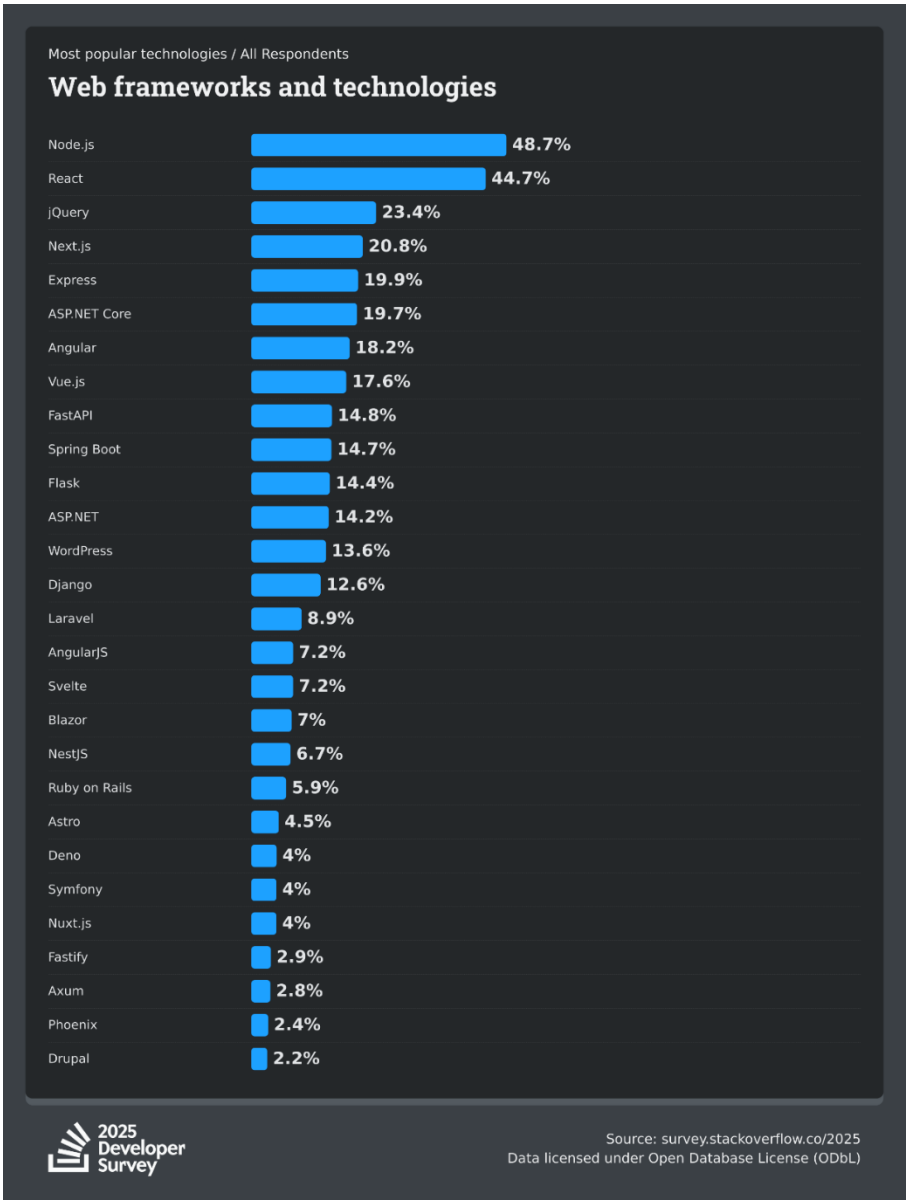



Figure 1: Most popular web frameworks and technologies, Stack Overflow Developer Survey 2025 [23].

From the Figure 1, one can see that React is the most popular frontend framework from all the previous that were mentioned with 44.7% of the respondents saying that they have used it in the previous year. jQuery is still the second most used framework even though it was released in 2006. At the time of writing this thesis, React package has over 70 million weekly downloads in the Node Package Manager (NPM) website [24].

### 2.5.1 React

React is an open-source front-end library and it is also often called a framework, and this thesis uses the framework term to discuss it. React was released by Facebook in 2013 [25]. It is currently the most used front-end framework [23]. It is especially made to create Single-Page applications (SPAs), and it uses JavaScript XML (JSX) to be able to use Hypertext Markup Language (HTML)-like elements inside JavaScript. This allows React to have the whole component's logic located in the same place. Components are individual, independent and reusable UI elements, that can be seen as HTML elements, but which also handle the JavaScript logic. These components allow React to respond to the user's actions dynamically. [26]

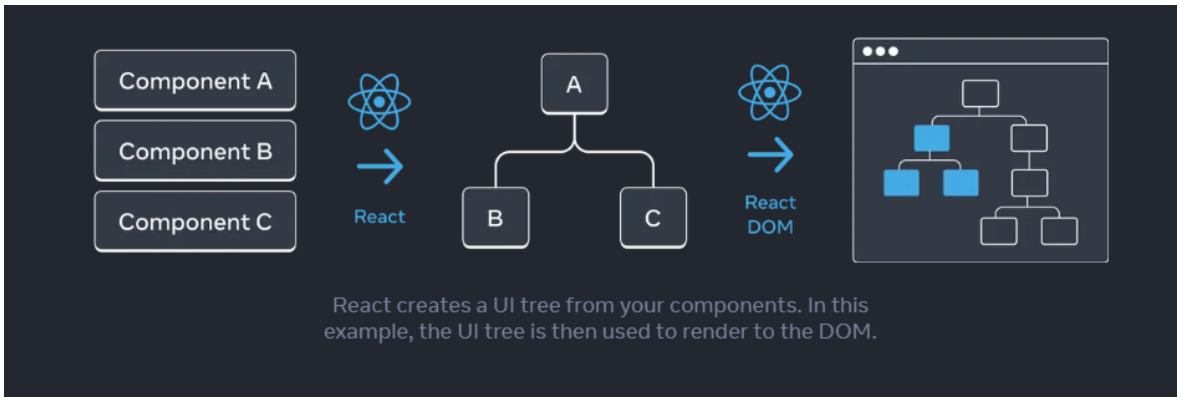



Figure 2: A figure of React render tree [27].

Behind all the code, React works by using a render tree to map out all the components into one single tree that displays the components hierarchy. It is used to keep a count which components are rendered, and which are not. This is especially important when you count in conditional components that have a specific condition to be rendered or not, based on for example the parent component's props. As seen in Figure 2, the render tree is used by React DOM to update Document Object Model (DOM) in the browser [27].

```tsx
import { useState } from 'react'
import './App.css'

interface ExampleComponentProps {  Show usages
    name: string
}

export function ExampleComponent({name}: ExampleComponentProps) :Element  {  Show usages
    const [count, setCount] = useState( initialState: 0);

    return (
        <div>
            <h1>Hello {name}!</h1>
            <button onClick={() :void  => setCount(count + 1)}>Increase the count!</button>
            <p>{count}</p>
        </div>
    )
}

```

Figure 3: An example of a React component with TypeScript.

The Figure 3 shows a very simple React component created as an example. It renders a HTML division element, a button and couple paragraph elements. The button has an event listener for clicks and when clicked, it increases the count state of that component, which is then displayed in the paragraph. The component also has properties, that are usually called props, which are like function arguments, that are rendered inside the JSX [26]. In Figure 3, the component only takes a name as props and that is dynamically rendered to the h1-element. From the Figure 3 one can clearly see the dynamic JSX component, where the JavaScript is used inside the HTML-like JSX elements.

React components are immutable, so whenever you want to change something inside of them you must create a new one. This is called re-rendering, and it usually happens if the component's state changes or the props change. In the component shown in Figure 3, it would re-render if the name was to be changed or if you click the button to increase the count state. The React state is a dynamic way to handle updating your component's data because it will always force a re-render, therefore it is one of the most used hooks in React. The state's value does not reset when a re-render happens. Hooks are a way to use different React features from inside the components and there are multiple official React hooks or one can also build their own custom hooks [28].

## 2.6 Server-Sent Events

In traditional web applications the data is sent and fetched via Hypertext Transfer Protocol (HTTP) or Hypertext Transfer Protocol Secure (HTTPS) requests. This allows the frontend (client) to request data with HTTP requests, and the backend (server) sends the payload what was requested. This works well in traditional web applications, where there is no need for the data to be sent as it is being generated on the server. This changes when we introduce an AI agent to the process. AI agents usually generate the data piece by piece, so it can be sent as an unfinished paragraph to the client, before it is complete on the server. This allows the user to read the response while it is still being generated. In this case the traditional HTTP requests do not work, because the client requests a conversation to be opened with the user's initial prompt, but after the first piece of data is sent from the server back to the client, it fulfils the request from the client's perspective. This means the client should send multiple requests to get the finished response that the server is generating. One solution for this problem is utilizing Server-Sent Events (SSE). It is an event-based unidirectional communication protocol which enables the server to push real-time updates to the client [29], [30], [31]. Because SSE is unidirectional, the client cannot send any events back to the server after the initial request. The communication is done via one HTTP connection. This removes the client's need to request multiple times and allows the server to send data continuously within the stream after the first request. [31]

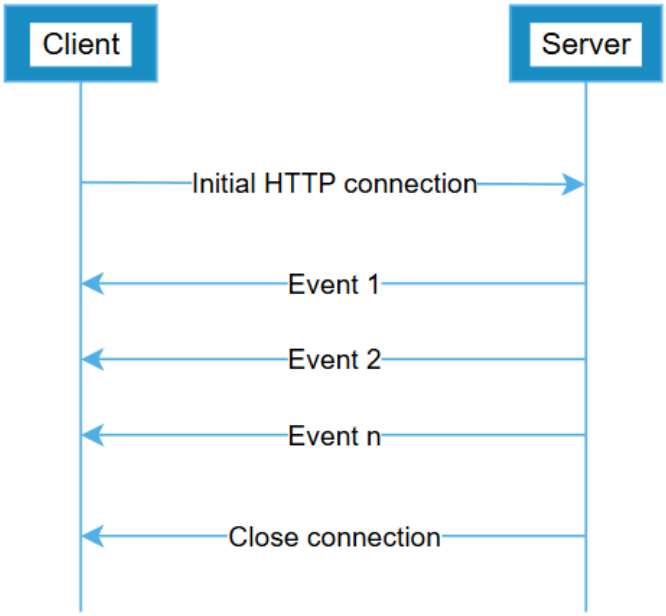



Figure 4: A sequence diagram of the Server-Sent Events flow.

The Figure 4 shows the sequence diagram of the flow between the client and the server. It shows that the initial connection request is the only request the client sends. After that the server will send its events and then close the connection eventually.

# 3 Research method

To comprehensively answer the research questions, an artifact will be created and evaluated. Developing and evaluating an artifact is one of the main points presented in research done by Hevner *et al.* [32] which describes design science especially in information systems (IS). Design science is described as a paradigm that seeks to create something effective through the artifact [32]. Therefore, design science is chosen as the research method and used in this thesis.

Design science is also characterized by Wieringa *et al.* [33] as creation and analysis of artifacts within their environments. The authors state that the process is an iterative cycle of solving design challenges and at the same time addressing knowledge questions. The authors explain knowledge questions to be questions that are evaluated by truth rather than depending on subjective stakeholder goals. [33]

This thesis adopts the Design Science Research Methodology (DSRM) based on the research done by Peffers *et al*. [34]. The authors present a design science methodology for information systems, and it was adopted by this thesis because it provides a structured framework for the presentation and evaluation of the research. It also acts as a road map of the research. It allows the building of an artifact and evaluating it based on the objectives given for it. [34]

Based on the DSRM by Peffers *et al.* [34], there are six key processes which are presented in the following sequence:

1. Identifying and defining the problem
2. Defining the objectives for a solution
3. Design and development
4. Demonstration
5. Evaluation
6. Communication

The DSRM research process can start from almost any of the six key processes. The starting points are the following based on the definition by Peffers *et al.* [34]:

1. Problem-Centered Initiation
    a. Starting from the identifying and defining process
2. Objective-Centered Solution
    a. Starting from the defining the objectives of a solution process
3. Design & Development Centered Initiation
    a. Starting from the design and development process
4. Client/Context Initiated
    a. Starting from the demonstration process

This thesis will start from the objective-centered solution, due to the problem having been already identified by the case company.

## 3.1 Identifying and defining the problem

The economic environment is pushing for the use of AI, and the year 2024 saw the biggest investments in AI by the private sector. Private investments to AI were $33.9 billion in 2024 and up by 18.7% compared to the year 2023. The use of AI in organizations is also rising globally, with the European organization's year 2024 percentage being 78% and up by 23 percentage points compared to 2023. This is indicative of AI being rooted into organizations and people's daily lives. [1]

The global trend of AI's popularity was noticed by the case company and the problem for the thesis was identified by them. This meant that the problem identification wasn't done during this thesis because it was identified beforehand. The case company had identified multiple problems, which had to be solved. The first problem was that the business environment of the case company was pushing for more AI features to stay competitive and relevant. The second problem was that the case company had identified that the lack of AI features was noticed by the customers and they gave feedback about it. Third problem, which was the basis for the artifact solution, was that the case application has a lot of features that

the users use daily, for example to track their work hours or create invoices. The case company identified that these repetitive daily tasks could be streamlined by using artificial intelligence to decrease the users' cognitive load.

## 3.2 Defining the objectives of a solution

The main objective of this thesis is to integrate a new artificial intelligence agent feature into existing PSA software to improve the user experience and to increase the case software's desirability from the customers' perspective.

The following objectives were defined for the identified problems:

1. To develop a user interface for an AI agent artifact, that reduces the users' daily cognitive load of repetitive tasks.
2. To measure the effect of an AI agent chatbot to user experience and user satisfaction in the context of PSA software.
3. To integrate a new communication method to support the AI agent response streaming to the existing application's architecture

The first objective focuses on the problem, which stems from the users' need for new AI features and the requirement for the application to remain competitive within the market of PSA software. The second objective measures the increase in user experience and user satisfaction within the software by introducing the artifact to solve the problem of daily repetitive tasks. The third objective addresses the technical aspect of creating the artifact, integrating new technologies into existing codebase and finding the best practices during the implementation.

# 4 Developing the artifact

This chapter focuses on the design and development process. Before starting out the development, there are some information that needs to be explained. There are two versions of the application, one that uses Inferno and one that uses React. The React application is the main application that is developed in this thesis, and it has its own component library that a Design System (DS) team designs and maintains. The frontend state management solution is shared between both versions of the application.

## 4.1 Development tools

The tools that were used for tracking and managing the project are explained in this chapter. During the development, the project was tracked through a project management application that allowed the teams to create and manage their tickets. The tickets were tasks that included design, development or testing work. Communication between the project stakeholders was done via a messaging application, and via a meeting application where live meetings could be had to discuss the project in more detail.

## 4.2 Initial architecture development

The initial planning process started with discussions about the designs that were made for the feature, and about the backend development process. These discussions revealed what kinds of components had to be used and what was already developed to the backend. The main idea was to create a completely new view for the AI agent. It would always be visible from the application's side navigation bar. The AI view was designed to have a prompt input component that is used to write and send the prompts. Then the prompt input component will include a prompt picker component that has some ready-made prompts for the user to pick from. The view also displays the prompts and agent responses in speech bubble components. Then there is a button to create a new chat. These were the basic components from which the whole feature was built on. The discussions together with the designers and

backend developer gave the general idea of the AI feature and how the user interface would be built.

The planning and development of the frontend architecture started after the initial discussions with the other teams. The main goal of this phase was to integrate the streaming capability of the AI agent to the existing architecture. This was a needed change to support the new text streaming capabilities, because there wasn't support for the streaming prior to this thesis and streaming the text as it is being generated is one of the main UI patterns in AI agents. There were some rules that had to be followed throughout the whole development process, one of which was that the coding standards and practices set by the development teams previously had to be followed. The new feature had to be integrated into the existing frontend architecture; therefore, it had to follow the principles of the existing architecture. The application's existing frontend architecture is a custom-made Flux type architecture. It follows the Flux architecture patterns, and the whole project has been written with JavaScript and TypeScript. All new code must be written in modern TypeScript to ensure maintainability and type-safety.

The custom architecture is built from stores, services, actions and one dispatcher. Views are present in the Inferno version of the application but are not that relevant in the React based application since the data layer and visual layer are not separated as strictly. As written in the Flux architecture book [21], the data-flow starts with the actions, that are called view actions in the application's architecture. These view actions are activated by user's actions within the UI. Then the view action is dispatched via the dispatcher to the stores that are listening for that specific action. The view action is only for one store, and it will be dispatched to only one store. The store then receives the information that an action it listens to has been dispatched. Then the store handles the view action accordingly, for example by sending a service call. If there are changes in the store data, the store will emit the changes so that the consumers of that store are updated accordingly. After the store has updated the data and emitted the changes, it will send out a server action. This is different from view actions, and the main difference is that view actions are made to be dispatched for a single store, the server actions are meant to be broadcasted to every store. The server actions can be used for example in complex model updates, to send updates to other stores that are dependent on the updated data. Server actions are also dispatched through the dispatcher, and stores have listeners for the specific server actions that they want to act upon with

handlers. The stores are created and managed with a store factory, that has methods for managing the stores. The whole state management architecture is relying on classes and inheritance and most of the basic logic has been introduced in the base classes. When the store is initialized, it will have data and state. Data is the current data loaded to the store and state is the current state of the store. The states can be used for example to inform the UI that the store is loading data, so the UI can show a loading indicator like a spinner or a skeleton. The services are handling the network requests between the frontend and backend. Most of the requests inherit a base service class, which has the required base logic built into it. This allows the child services to use the base methods. The services support only traditional HTTP requests.

The data streaming feature was planned and there were couple approaches which supported the streaming capabilities that were needed. WebSocket and server-sent events were studied for the implementation of the text streaming to the UI. WebSocket makes a two-way connection between the browser and the backend, and they can communicate to each other before terminating the connection [35]. The server-sent events work unidirectionally, and the browser only sends one request to the backend while the backend sends the n -number of events back before terminating the connection. The existing service logic already used regular HTTPS requests together with JavaScript Application Programming Interface (API) called XMLHttpRequest (XHR). Even though XHR includes the type XML, it is not limited to only XML types of data [36]. On top of that, there was no need for the browser to send any data, other than the initial request to the backend. Server-sent events were chosen as the communication method that would support the needs for the text data streaming because it followed the coding standards that were set. The support for the server-sent events had to be built as a custom solution to be fully integrated into the existing network architecture. Before starting to implement the server-sent events support, the content of the stream body had to be decided together with the backend developer. The decision was made to follow the server-sent events standard body, which has an event and then on a new line the data and these combinations are separated with empty line.

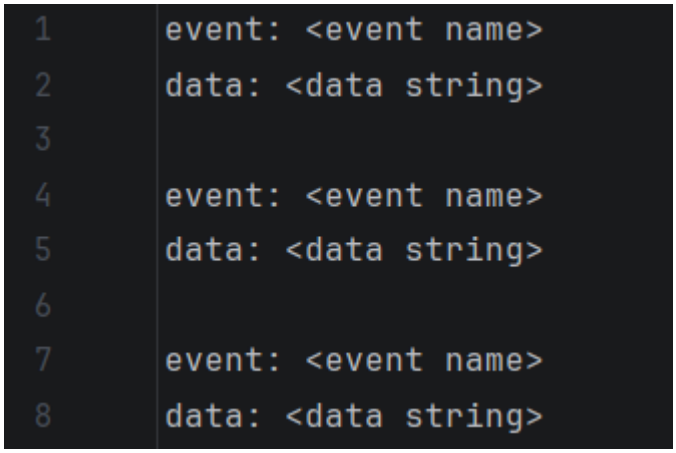



Figure 5: Server-Sent Events format.

The SSE type format chosen for the case application can be seen from Figure 5. The format is the standard way to handle streaming content, and it makes it easy to parse in the client instead of just sending strings without any unified format. Event line specifies the type of event that is sent and data line is for the text content [37]. The fields are separated by one newline, and the complete data chunks are separated by two newlines like in the standard SSE format [37]. In the case application the streaming response body is a string payload which then will be parsed into a JavaScript Object Notation (JSON) in the application.

## 4.3 Creating the service

A new service had to be created to send requests to the new backend routes which were created for the AI agent. The AI logic was built by the backend developer in a separate AI microservice, that the backend was going to be connected to. During the initial development phase, the service was connected to the AI microservice instead of the application's own backend. The AI microservice had to be used directly to allow the UI development to start sooner because the application's backend was not connected to the AI microservice at this point in the development. The new service initially had one method, which handled sending the initial requests to the server. It used an already existing authentication helper function from the base class. This helper function checks the authentication token before sending a network request. To get the new service to work like intended, the network layer had to be modified. It didn't have support for any kind of streaming or server-sent events. The network layer was modified to have a new function. This function handles the whole

XMLHttpRequest and it relies on the XHR status codes to work during the streaming process. It gets new data chunks from the XHR response text property and makes the necessary checks to ensure that there is new content and then it calls a new data processing function with the data chunk, a stream chunk event handler and a success callback function. This way the new AI specific store will get the data from the network layer in chunks that are parsed into JSON format. This works because in server-sent events, the data chunk that is streamed, also includes the previous data chunk, so the new data is appended at the end of previous chunk.

## 4.4 Creating the store and actions

A new store had to be created for this feature. It extends a base store class to get the basic base functionality. The AI store constructor initializes the data, which initially stored message, agent tool call results, the tool calls and error. Later during the development, the data included only message, thinking, error, actions and conversation info.

Before moving on to explaining the actions and development, the store states must be explained. The stores have states, which can be set and removed to reflect the current state of the store. In the context of AI agent, the new AI store first had to send the initial request to the server, which then started the streaming process using the server-sent events. This initial connection was handled with setting a loading state to the store to indicate that the state is loading, not streaming yet. Then after the server starts streaming the data back to the client, it is called streaming. A new state was created to support streaming, because there wasn’t a correct state for these kinds of features before. These states ensured that the UI knows the difference of loading and streaming, which is necessary for the prompt input component to show the different states correctly.

The store was made to handle two custom made view actions initially. One was to post a message, and the other one was to stop the stream. The post message action was needed for the user to being able to start or continue the conversation. The stop action was needed for the stop functionality so that users could stop the answer from being generated if they wanted to. Both actions have handler functions that handle the actions once they come from the dispatcher to the store. The post message action calls a function inside the store which takes in the data, a success callback and action options as parameters. This function makes sure

that when this view action is dispatched, that the store will go into loading state, to indicate the current state of the store and then it sends the necessary information to the AI service's post message method. The request is then asynchronously handled with the JavaScript's then and catch, and it waits for the request to successfully complete before calling the success callback. At this point the error handling is done via JavaScript's catch statement. If an error happens, the store calls the base store's error callback which handles the error on the store level, and it will display a notification toast automatically in the UI. There is also an always clause, which is used to always remove the loading and streaming state from the AI store. The states are removed to ensure that the store will never be stuck on those states in cases of errors or successes.

The method which sets the store to streaming state is the store's stream chunk event handler. This method was created to handle the data flow from the chunk into the store's data. It takes in the raw text chunk as a string parameter. The string parameter is already processed to include only the chunk's data string and nothing else from the server-sent event data chunk. The function then parses the text data chunk into a JSON object and extracts the data from it. The data is then set to the stores data. The tool call results and tool calls are sent only once from the server to the client to minimize the streaming data size. The function checks if there are any changes and adds them only if it detects changes in the data. This function is called every time there is a new chunk that arrives from the server to the client.

There is one more method inside the initial AI store. A private method, that handles the case when user wants to stop the streaming mid-stream. It is called when the store receives the stop view action. It aborts the request, which then stops the streaming. It removes the loading and streaming states after aborting the request.

## 4.5 Developing the initial UI

With these new additions to the network, store and service layers, the UI development could be started. During the architectural changes, a mock UI, which didn't use any of the styling or components from the designs, was used to test out the store logic. When the UI development started, a new route had to be created into the app's router. The new route was created, and the route for it was decided to be /ai.

It is necessary to understand the application and how it utilizes feature toggles. The developers can use feature toggles to show or hide features from the users. The toggles are a great way to limit the visibility of the features so that the features can be tested with a specific group before all the users get access to it [38]. Feature toggles were also used in the AI agent development process to prevent the users from seeing the feature before it is ready and tested. This meant that one had to be added to the feature toggle list. The whole route navigation was set behind the feature toggle, to ensure that users couldn't access the AI agent during the development process. The toggle was needed because the UI application is released to production frequently.

In the case application, all the stores are used via custom made React.JS hooks, which abstract the store handling from UI to the data layer. The hooks have the basic logic to call view actions and to manage the data in a React state, to ensure that the data refreshes in the UI. The initial view utilized these hooks to manage the data from the AI store. A new custom React.JS hook was created that used one of these store hooks to abstract some of the actions down to just single simple function calls, that were exposed to the UI from the hook.

When the new route and navigation changes were done, there had to be a view that the /ai URL takes you. The initial view had to be created in the UI, and it was a simple view with a text area for the messages and a prompt input. In this step the design system components came into use. The components that were created by the design system for this feature were the prompt input, prompt picker, speech bubble and a thinking component. The speech bubble was not ready to be taken into use for the initial view, so the conversation view had to use plain text instead of the speech bubbles. The initial view was then done; it had a conversation window for the texts from the user and from the AI agent and a prompt input. This view was used as the base for all future development to be built upon.

When the UI development started, the backend wasn't ready to be used yet. It was decided that the frontend uses the AI microservice directly during the development. This meant that all the conversations were stored to the browsers session storage to ensure that the ongoing chat did not disappear during the use of the application in the case that the user navigated away from the AI agent view. Because the UI was developed at the same time as the backend and it had not been decided if the backend conversation history would be in the scope of the feature, the post message calls had to include the chat context history to be sent along with

the new messages. This kept the AI agent up to date with the context because AI agents are usually stateless and require the context to be sent in every call.

## 4.6 Improving the user experience

The initial version of the implemented AI agent was quite simple; it did not contain any animations or actions that the user could take. The user could only prompt the agent in a conversation-like manner, and the agent could respond with text. The first improvement to build upon the working base was to add animations for the agent's response. This is usual and popular pattern in the most used AI agent conversation tools, which are streaming the answer as it is being generated and not as one big wall of text once everything is complete even though it might be seen as a dark pattern by some [39]. Before the improvements, the UI displayed the agent streaming the answer, but the answer just appeared in chunks and was dependent on the network speed because the chunks were displayed instantly when received. The animation that was added improved the experience a lot by allowing the response to be read while it is still being streamed with a smooth animation. All the components were not ready for the working base version of the AI agent so the speech bubble for the messages in the conversation and a prompt picker were added in the later development phase. Adding these components improved the look of the feature and the prompt picker sped up the user flow with its ready-made prompts that are available for the user to choose and send to the agent. The initial prompts were constructed by collecting feedback from the project owner, designers and developers and then the prompts were ranked from priorities 1 to 5, with 5 being the most important. Then all the prompts that were ranked with the highest priority were chosen initially to be included in the prompt picker during the first testing phase.

The animation that was added needed to take into consideration that the user could stop the stream at any point and that the response text is ready at different times depending on the network speed and response size. This meant that displaying the streaming could take a lot of time if the network speed is slow and if the message is long. To combat the animation taking too long, it was decided that it adapts to the message's length. A custom React.JS hook was built to handle the animation. It takes three arguments, which are the current streamed text, a Boolean value to know if the stream has stopped and a state change listener

function. These arguments had to be included for the animation to feel responsive as well as make the animation faster when there is a lot of text to be animated. This prevented the case when the answer would already be received as complete sentences, but the animation would be still playing. The state change listener is a callback function which handles the animation state from the parent view to ensure that the parent view always knows the animation state. With the addition of the animation, there was a requirement to ensure that if a new message was sent and a stream of text is streamed in response, the conversation window will always display the new messages even if there is a vertical scrollbar. This was handled with another custom hook, which automatically scrolls down to the newest message if the content would overflow. It keeps the chat responsive and improves the user experience. The automatic scroll hook also checks before scrolling, if the user is near the bottom and only scrolls to the bottom if the user is near it. This was done to prevent the use-case when a user wants to read previous messages when the agent is answering. The speech bubble component, which was taken into use, had actions that had to be added. Two actions were designed, one to the user's message and one to the AI agent's message. An edit functionality was added to the user's own messages, so that the user could edit the prompt if it wasn't what they wanted to ask but had already sent it. The AI agent's message action was a redo action, which sent the same user prompt again to the AI agent to get another answer. This is important because sometimes the AI agent could answer wrong and then the redo action is helpful to prompt the agent again for a new response.

## 4.7 Refactoring the services

The feature had to be initially built to call the AI microservice instead of the backend and it relied on storing the current conversation to session storage. This was because of the earlier design choices, where the backend conversation history was not scoped into the feature. This meant that when the backend was ready, a major refactor had to be done to change the route from the AI microservice to the backend API route. During the refactor, the project team also decided that the AI agent should have the conversation history features in the backend. This meant that while refactoring, the functionality had to stay the same as before as well as adding the conversation history to be loaded from the backend instead of session storage. This caused problems that could have been avoided if the conversation history was designed to be implemented from the start.

The new AI agent conversation history used the URL parameters to keep track of the current conversation and its Globally Unique Identifier (GUID). This is the same pattern used in every other view in the application. A new list store had to be created for the single conversation history. During the implementation of this store, there were multiple issues with trying to integrate the new kind of feature into the existing architecture because of the nature of the list store that had to be implemented and the user flow being so different compared to existing features. Firstly, the history store should not fetch during the initial conversation, because the streamed response from the agent already includes the data that is needed to show it for the user. The list store should only fetch when the GUID is changed from one conversation to another or when the user navigates from a history list menu to a conversation. The new list store had to overwrite the existing base fetch methods, so that it would not fetch during the initial conversation, because fetching during the initial conversation start caused unnecessary flickering in the UI. The flickering happened because the history store would go into loading state during the stream which then caused the UI to display a different component than it should have. The fetch method was overwritten so that it checks the conditions and only if the correct conditions apply, it shall fetch the conversation history from the backend. This prevented fetching with null GUID and fetching during the initial conversation. While there were some problems with refactoring the history to use the backend route and the conversations being loaded from the server, the conversion from the AI microservice to backend API was quite simple to refactor due to the service architecture.

## 4.8 Transparency of the AI agent

To ensure transparency and to ensure the user feels like they are in control, the AI agent was designed to include a thinking component from the DS component library. AI agents need to be as transparent as possible to increase the user's trust in the system [40] and allowing the user to see why the AI agent responded in the way it did can increase it. The component includes all the reasoning and thinking information that the agent used to come up with the response. This ensures that the users are not left to wonder why the agent answered in a specific way, especially if the answer is wrong.

## 4.9 Creating the work hour actions

The last missing parts of the feature that were designed and planned at the start were the actions for creating, editing or rejecting the workhour suggestions. First the add work hour action was created. This action laid the foundation for all the other actions. The actions were planned to send two API service calls. The initial API service call targets the application's existing work hour route, which creates the work hour with the given data, and the second one sends a PATCH operation call to the conversation history route that ensures the entries are displayed with correct information in the chat history. The other option was to call the backend with single API call, but it was decided that two calls are better in this case, due to the calls not being that tied together therefore atomic operations were not needed. It was also justified to reduce the scope of this project.

The add work hour action uses the already existing stores and store hooks, which handle the work hour creation. The existing architecture did not support the new PATCH operation, which could patch inner object models inside the conversation history, and therefore a new helper function to handle these kinds of cases had to be created. A new method was created into the AI service which supported the PATCH operation as well. The order of requests was decided to first add the work hours and then send the PATCH operation to patch the conversation history in the success callback function. This way the conversation history will never be patched if the work hours are not created due to an error. After the add action was created, the editing and reject actions were straightforward to add on top of the new existing code which already supported the editing and rejecting actions, only the UI had to be implemented.

## 4.10 Additional features

During the implementation of the AI agent, the developers and designers decided to add some features that were not designed to be included in the initial versions. Most of these features were added to improve the user experience and some of these were required to be implemented to follow local regulations.

Initially the conversation history wasn't supposed to be stored in a database but since it was decided to include it in the scope, it meant that a conversation history list should be added

as well to let the users view their previous conversations. The conversation history list was also decided to be implemented due to the already existing backend support for it, which was necessary to implement to support the other features. While developing the conversation list, there was a need to have the previous conversations deleted. This was because of General Data Protection Regulation (GDPR) article 17 right to erasure, says that the user must be able to delete their personal data [41]. The deletion of previous conversations was then implemented, and it utilized a simple DELETE request to the backend. The delete feature was not present during the internal testing because of time constraints.

# 5 Demonstration

In this chapter, the feature will be demonstrated and shown how it works in practice inside the case application. The UI was simplified to display only the AI agent feature. Other features inside the case application were removed for this chapter because this thesis focuses solely on the AI agent. This can be seen in Figures 6 to 10, which display the UI in a simplified state, only displaying the relevant AI agent interface. The demonstration chapter includes the main user flow of requesting and adding work hours from the AI agent.

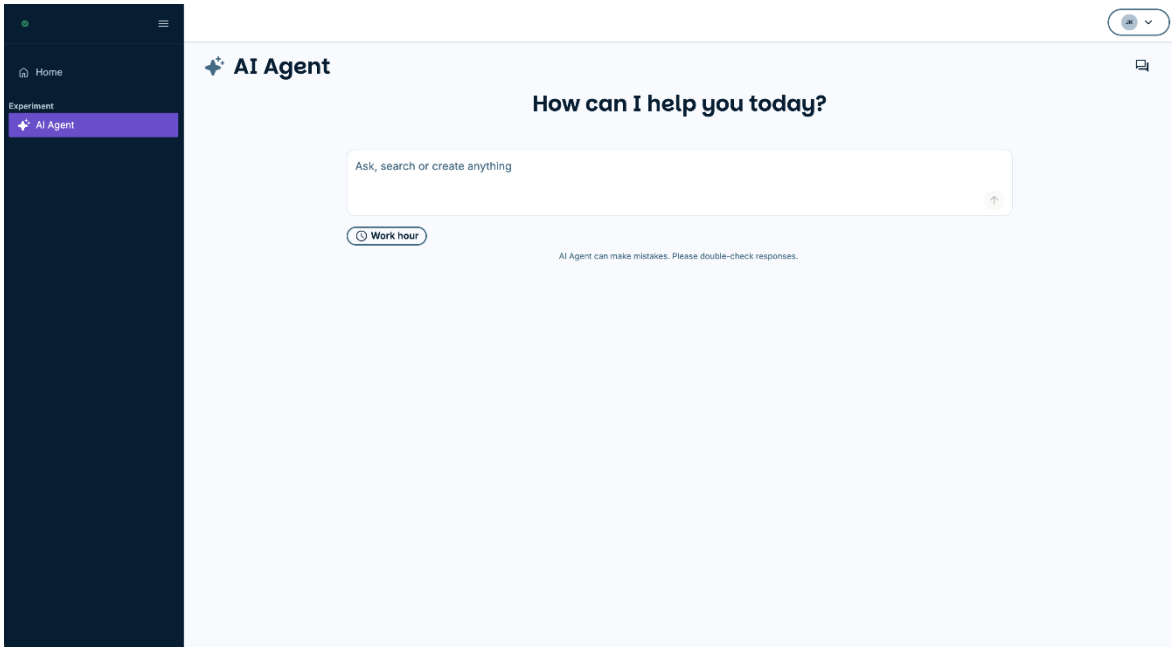



Figure 6: Landing page of AI agent.

Starting from the initial landing page that the user will navigate to at the start. It can be seen from Figure 6, and this is the page the case application's router will navigate the user when they navigate from the side navigation to the "AI agent". Here the used DS components can be seen like the prompt input, which is the input field for the prompt as well as the prompt picker that can be seen under the prompt input with the text "Work hour". The conversation history button is in the upper right corner, and it opens a drawer component that shows the user's previous conversations.

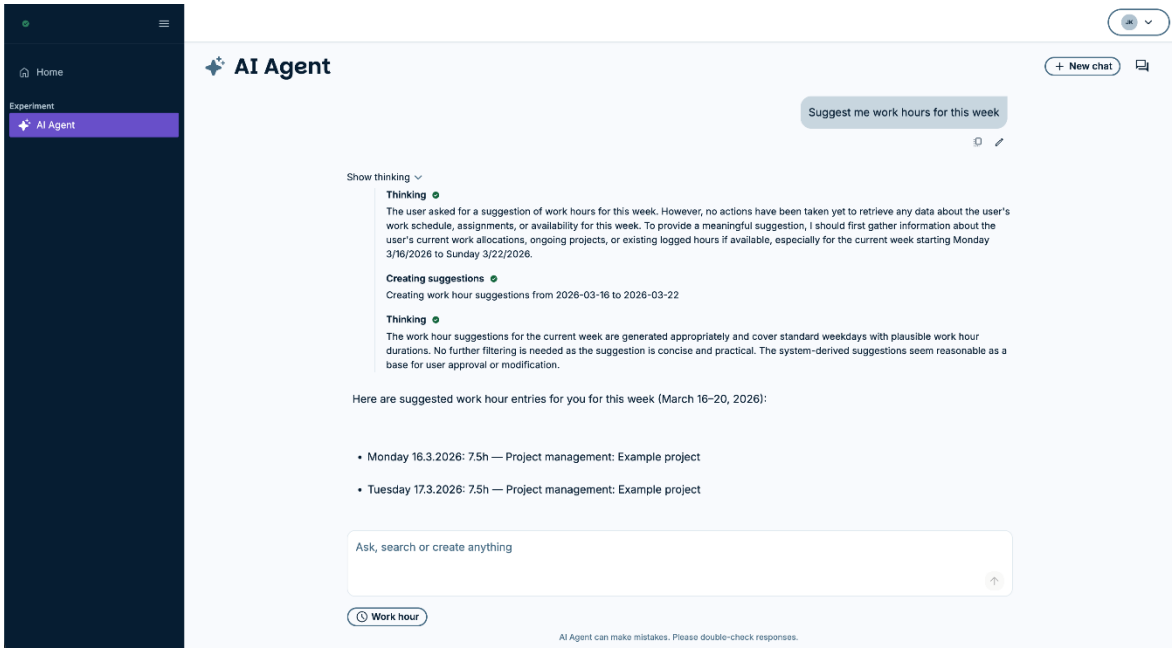



Figure 7: View after prompting the AI agent with a request for work hour suggestions.

After the user arrives to the landing page and sends a prompt to the AI agent, they will get a response that will be displayed like in the Figure 7. The response includes the thinking component which shows the AI agent's thought process and speech bubbles as the user's question and AI agent's response text. The thinking component is collapsible, and it is collapsed by default but in the Figure 7 it is opened. In the Figure 7, the user has prompted the AI agent with a request to suggest work hours for the week, and the AI agent has thought three thoughts. The text response can be seen under the thinking component where the AI agent has responded with a text to the user's request. In the upper right corner, there is also a "New chat" named button, which navigates the user back to the initial landing page of the AI agent. This button appears only when the user has navigated to a conversation.

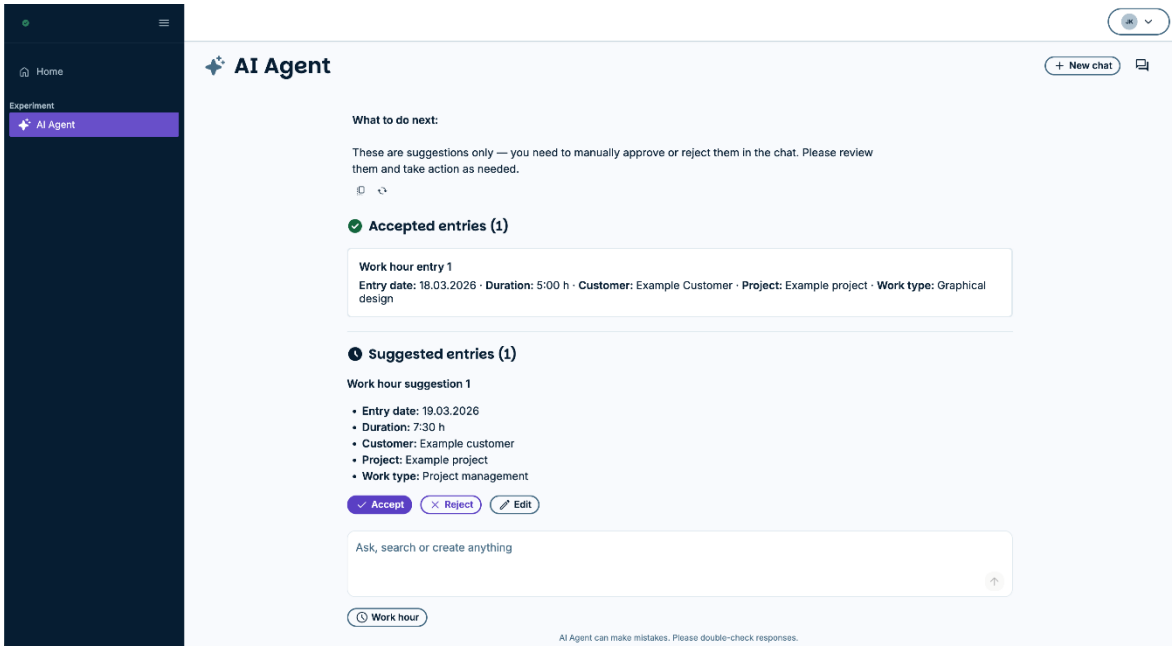



Figure 8: Accepted work hour entries and work hour suggestions, some of the suggestions have been rejected already by the user.

After scrolling down within the same conversation, the user sees the suggested and accepted entries. In Figure 8, there is one accepted entry and one suggested entry, meaning that the user has already accepted one entry. The accepted entries are shown in the conversation as list item components, that include the necessary information about the accepted entries. In this case the entry date was for 18.03.2026, duration was 5 hours, the customer was “Example Customer”, and project was “Example project” with the work type being “Graphical design”. If there are no accepted entries, this list and its title would not be rendered. Under the accepted entries the user can see the suggested work hour entry that the AI agent has created as suggestion. The work hour suggestion is for the date 19.03.2026 and the duration is 7 hours and 30 minutes, which the AI agent derives from the work hour history that the user has, the project and customer are the same as in the accepted entry and the work type is “Project management”. The user can accept, reject or edit the suggestions. When the user accepts the suggestion, a success notification toast will be shown at the bottom of the application screen, and the entry will be moved to the accepted entries list. If the user rejects the work hour suggestion, a danger variant information toast will be shown at the bottom of the application, and the suggestion will disappear from the UI. When editing a work hour suggestion, the UI will render an edit form for that work hour suggestion.

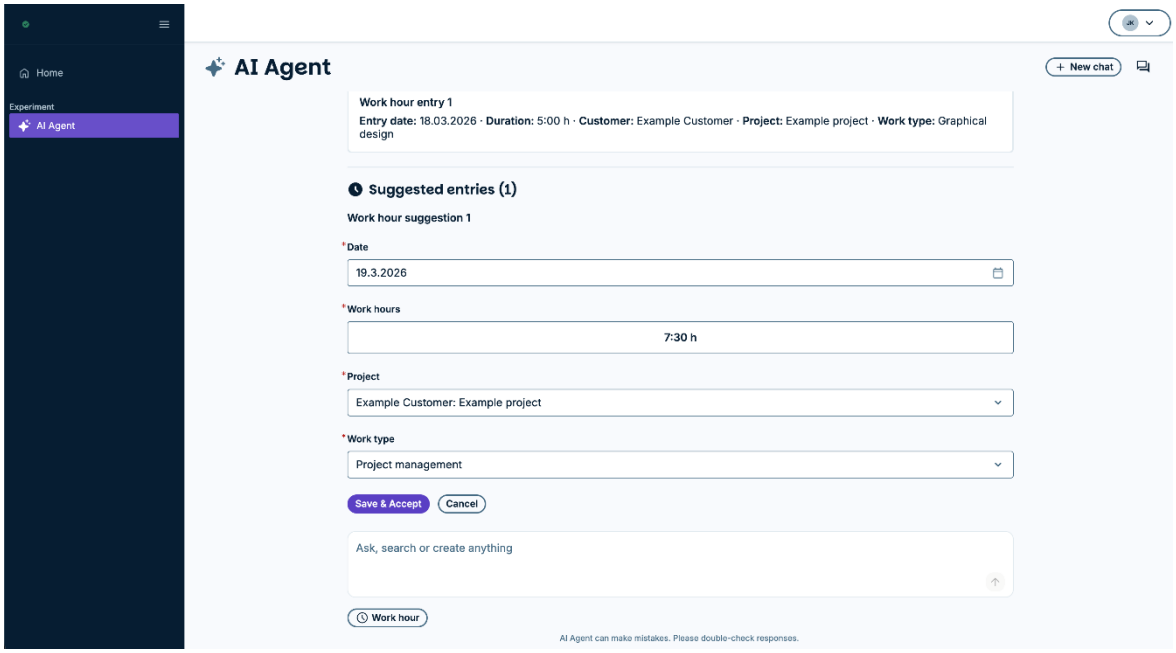



Figure 9: Edit work hour suggestion form.

When the user clicks the edit button, it will open an edit suggestion form as seen on the Figure 9. The user can edit the following fields: date, work hours, project, work type. The date input is a date picker, and the work hours field is a specially made work hour input that is based on a text field component, and it supports the hour format. The project and work type fields are both select components. The user can modify the suggested entry data from this form and then either save and accept it or cancel it. Save and accept will accept the work hour and cancel will reset the work hour back to the original state and close the edit form.

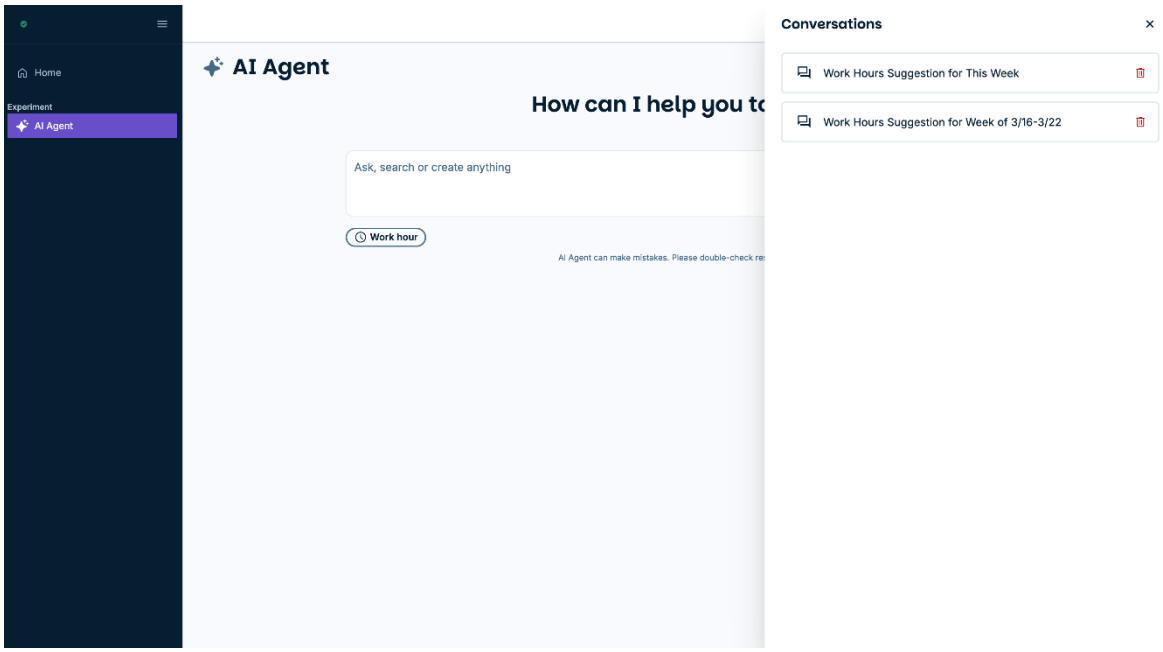



Figure 10: Previous conversations drawer.

When the user clicks the conversation history button from the top right corner, a conversations drawer will open. The drawer displayed in Figure 10, displays the user's previous two conversations. The drawer would display the current conversation that the user is viewing with a different background colour, but in the Figure 10 the user is at the main page, so no conversation is selected.

# 6 Evaluation

Evaluation phase utilizes survey research, specifically an explanatory survey as defined by Wohlin *et al.* [42]. It aims to explain why and how an artifact affects the target population by examining the relationships between different variables. This type of survey was chosen to understand how the artifact affects the user experience and how does the AI agent compare to a traditional user interface in its usability.

The survey uses convenience sampling, which means that the sample consists of ready and available individuals [43]. The survey participants are the case application team members who have participated in the internal testing of the AI agent. Convenience sampling was chosen because the feature had to be tested internally to ensure data collection was done on time for this thesis. The access to real users testing the feature was precluded due to these limitations. Therefore, the results might not correctly reflect real customers' experiences. Even though the internal testers use the case application to enter their work hours they might have a more complete look and understand the features of the application in a different way compared to traditional normal users, who do not work with the application daily.

The survey consists of four specific sections: background information, AI agent specific questions, the System Usability Scale and open-ended free text questions. The background section has baseline questions which will give the baseline data for this survey. AI specific questions and the SUS utilize Likert scale, which is from 1-5, 1 being strongly disagree and 5 being strongly agree. More details about the survey can be found from Table 1.

System Usability Scale was chosen for the survey over the chatbot-specific scaling, like BUS-11. This decision was made because the focus of this thesis is to evaluate the user interface patterns and usability of the AI agent instead of the AI orchestration logic. The survey was created with Google Forms, and it was shared to a Slack channel that had all the case application's employees present.

Table 1. Survey sections and their questions and scale

| **Section** | **Question** | **Scale** |
|---|---|---|
| Background | Experience with case application | Never used it before testing the AI agent,<br>I use mostly basic tasks like marking work hours,<br>I use it for complex tasks like reporting,<br>I use most of the features and can train others to use it |
| | Familiarity with AI tools (ChatGPT, Gemini, etc.) | I have never used one,<br>I have used them once or twice,<br>I use them occasionally,<br>I use them daily |
| | How often do you use case application? | Monthly,<br>Weekly,<br>Daily |
| AI agent specific questions | Adding work hours from the work hour -view is a time-consuming task. | Likert scale |
| | Adding work hours from the new AI agent is a time-consuming task compared to the existing work hour -view. | Likert scale |
| | The AI agent's thinking element helps me to understand what it is doing. | Likert scale |
| | The animated response allows me to read the response before it is complete. | Likert scale |

| | | |
|---|---|---|
| System Usability Scale (SUS) | I think that I would like to use AI agent frequently | Likert scale |
| | I found the AI agent unnecessary complex. | Likert scale |
| | I thought the AI agent was easy to use. | Likert scale |
| | I think that I would need the support of a technical person to be able to use AI agent. | Likert scale |
| | I found the AI agent functions were well integrated into case application. | Likert scale |
| | I thought there was too much inconsistency in AI agent. | Likert scale |
| | I would imagine that most people would learn to use AI agent very quickly. | Likert scale |
| | I found AI agent very cumbersome to use. | Likert scale |
| | I felt very confident using AI agent. | Likert scale |
| | I needed to learn a lot of things before I could get going with AI agent. | Likert scale |
| Open ended | What did you like most about AI agent? | Free text |
| | What was the most frustrating part of using AI agent? | Free text |

## 6.1 Results

The survey was open for seven days and it received seven responses. The response rate was only about 8% of the sample, which is quite low. The survey participants were all part of the case application's employees. This means that the results cannot be generalized, but the open-ended questions can be used to draw conclusions about the tested version of AI agent and suggest improvements to it.

First the background questions are analyzed. Overall, the participants familiarity with the case application was good and almost everybody uses other AI tools daily. Five participants answered to use the case application for only basic tasks. Only one used the case application for complex tasks and one used most of the case application's features and could train others. The responses indicate that the survey participants are very familiar with other AI tools. Six out of seven responses say that they use other AI tools daily, where only one says to use the AI tools occasionally. The survey participants use the case application often, where 4 participants report using it daily, 2 say that they use it weekly and only one reports monthly use.

The survey participants are neutral about the current implementation of the work hour view with the average score from the Likert-scale being 3. When comparing the work hour view time consumption to that of the new AI agent's, the average score was 2.71. This result is below neutral, slightly towards strongly disagree end of the Likert-scale. Participants thought that the thinking element helps them to understand what the AI agent is doing. This can be seen from the average score of 4.29, which is slightly under 5, meaning strongly agree. The animated response helping with reading the response while it is still generating was also seen as helpful by the participants, the average being 4.43.

Moving on to the SUS scores, which consist of the ten pre-defined standard questions. The SUS rating is calculated by taking the 10 questions and calculating the score contribution for every participant and then summing those and multiplying the summed value with 2.5. The odd values are calculated with a formula [Odd value - 1] and the even values are calculated with [5 – Even value]. [44] This gives the individual score and then the system rating is the average score of the individual's scores. This system scored 68.21 on the SUS rating.

Both open-ended questions were answered by five participants. These responses are analyzed more in detail in the discussion chapter of this thesis. The results of the survey can be found from Appendix 1.

# 7 Discussion

This thesis created and integrated an AI agent artifact into existing PSA application and used a survey to gain an insight into the usability of this feature. This chapter first discusses the results of the survey and then the discussion moves to the artifact and especially the research questions. At the end of this chapter there is discussion about realized risks and limitations of these findings.

## 7.1 Survey results

The survey background questions mapped out the participant's background knowledge on the case application and on other AI tools. Most of the participants are basic users of the case application who use the application daily. Most of the participants also use other AI tools daily, which suggests high knowledge of the basic AI features and user interface patterns.

The survey results indicate that the AI specific elements were well received by the participants. The responses suggests that the thinking component adds value and helps the users to understand why the AI agent has come up with the response. Using the thinking component could be interpreted as increasing the user's trust and the user's perceived autonomy by showing the agent thought process and combatting one of the identified problems in the paper done by Nguyen *et al.*[15] where chatbot interfaces leads to lower perceived autonomy. According to the results, most of the users agreed that the agent response animation allowed them to read the response before it was completely ready. This response animation element can be seen as making the AI agent more human-like by showing the response while it is still being made. The score can be explained with the animation increasing the agent's human-likeness and how the non-experienced application users preferred the agent to be more human-like in a study done by Gnewuch *et al.* [17]. There were still two answers in the open-ended questions, that criticized the slowness of the work hour suggestions, which indicates that those participants might be more experienced with AI tools.

The SUS rating the system received was 68.21, which can be pinned down to a C- or D grade according to the previous research [8]. This means that the system's usability is on the verge

of being average, so it is not awful, but it is not exceptional either. This rating can be explained by some of the open-ended responses, where the system is said to have hallucinated answers and work hour suggestions. The AI agent was still perceived as easy to use with the average being 3.86 and to be easy to learn with the average score being 4.00. This is aligned with the results where most of the participants are familiar with other AI tools, indicating that the AI agent follows the basic standard user interface patterns which make the system easy to learn. This is backed by one of the open-ended answers where the participant states that the UX patterns are familiar. According to the survey data, some of the participants would use the agent more frequently, the average being 3.43 but still closer to a neutral answer.

Most of the frustrations found from the AI agent were towards the agent logic itself, where the agent had hallucinated and lied to the user. The longest response to this open-ended question indicates that the AI agent lied to the user in most of the responses, which might have caused the other scores to be lower in the SUS questions. Even though this thesis did not focus on the AI agent logic, this is a clear place for improvement before the agent can be released to production. The comments suggest that the slowness of the system could be improved with clearer indication of the agent still being in streaming or thinking mode, when creating the suggestions. One of the flaws noticed by one participant was that they found it hard to jump around the AI agent and the traditional work hour entry view. This could be due to the user trying to check that the work hours are created, but there are also improvements to made in the future where the AI agent and the traditional view could be used in the same view, like the study done by Nguyen *et al.* [15] which suggested to combine the traditional menu-based interfaces with the chatbot interfaces.

Even though there were some frustrations, some of the participants shared notes about the things they liked about the AI agent. One response mentioned the look of the AI agent, where they liked the modern look and one mentioned familiar UX patterns that they liked. Three of the participants mentioned that they liked the work hour suggestion tool, indicating that the actions are useful for some of the case application users. One of the participants liked the data retrieval ability and the agent using the retrieved data in its responses.

Overall, the survey data suggests that the user interface and user experience of this system was average based on the SUS rating but well received based on the open-ended questions. The positive notes were about the work hour actions and UI and UX, whereas the noticed

flaws were mostly about the slowness of the system as well as the AI agent hallucinating wrong answers and lying to the users. The two UI/UX elements used, the thinking component and the response animations were well received and should be studied more with larger sample size and with real users to really see if they are highly usable UI components in chatbots.

## 7.2 AI agent

During this thesis an AI agent feature was introduced into the case application. It was developed successfully and based on the survey results; it had some positive influence on the users' daily load with repetitive tasks of adding work hours. The initial scope of the feature was changed multiple times during the development, which made some of the earlier design choices unnecessary. It also meant that a lot of refactoring had to be done, for example with the conversation history and its implementation. This is something that could have been prevented with better planning with the designers and developers, but it did not slow down the development process as the feature was ready on time for the internal testing.

For the AI agent to work properly, a new communication method had to be introduced to the existing network architecture. The new communication method was chosen to be server-sent events, which was deemed necessary to follow the standards and previous architecture. Compared to the other available technologies that allowed streaming like web sockets, server-sent events proved to be easy to use together with the existing XHR network call architecture, because of its suitability with regular HTTPS requests. Therefore, the SSE was integrated into the application's architecture, and it followed all the standards and practices that were required. The key takeaways of integrating AI features into an existing architecture is to follow the previous coding standards and practices. This was noticed very early on during this thesis when thinking about different ways to implement the new AI feature. Therefore, most of the technical decisions that were made during the development of the artifact, were heavily influenced by these factors.

Before starting out the development of the artifact, three objectives were laid out for it. The first one being to develop a user interface for the AI agent artifact that would reduce the user's cognitive load of repetitive tasks. This first objective was achieved by developing the feature and by surveying the users who tested the feature. The survey results indicate that

the feature is usable and the work hour actions were liked by the participants, which replace the repetitive task of adding work hours from the traditional menu-based interface. The second objective was to measure the effect of AI agent chatbot on user experience and satisfaction. This effect was demonstrated with the survey results, where the effect was mostly positive in the context of UI and UX. The third and last objective was to integrate a new communication method to support the AI agent response streaming to the existing application, and this was demonstrated and explained to have been achieved by creating the artifact and the new support for the server-sent events within the network layer of the case application.

## 7.3 Research questions

This thesis had three research questions which were 1) What are the technical best practices of integrating artificial intelligence tools into existing software architectures, 2) How do different user interface patterns for agentic artificial intelligence affect the usability and error rates in professional services automation software, 3) Which user interface patterns provide the highest usability for users using an artificial intelligence conversation tool in professional services automation software. This thesis argues that the best practices are to follow the existing coding standards and guidelines which are set by the teams and researching modern methods that could be integrated and used together with these standards and guidelines. The agent response animation pattern was met with positive survey results, and it improved the usability by increasing the human-likeness of the agent. The survey results also indicate that the actions from the chat are easy to use and lessen the user's cognitive load when the action can be accepted without editing. The thinking component can help with the error rates, as it shows the user what the agent thought as it did wrong actions. The main way to show errors that happen during the API calls within the system was to show a notification toast, but the toasts did not improve the usability with agentic errors that stem from logic mistakes where the thinking component could help in the future.

## 7.4 Realized research risks and limits

There are risks and limitations that must be considered when discussing about this thesis and its findings. The first one is that the development work was split into two, where this thesis focused on the frontend development and the backend development was done by another employee of the case company. This means that some of the survey results might be influenced by things that this thesis had no control over. The deadline of this feature and the need to release the AI agent to production also influenced the quality of the agent responses, because of the time limits not allowing the backend developer to enhance the agent logic before internal testing. This can be seen in the internal testing survey responses, where most of the open-ended notes where the participant was asked about the negatives are mentioning the logic of the agent instead of just the user interface.

This thesis develops and analyses only one application and the integration of artificial intelligence to the one application, therefore it cannot be generalized to a broader view of applications. Due to the low survey response count of seven, this thesis cannot draw any conclusions that could be generalized to other agentic AI features or applications that want to integrate artificial intelligence into them.

# 8 Conclusions

This thesis successfully integrated a new communication method and a new artificial intelligence agent feature into the existing PSA application. The main research questions were about the technical best practices of integrating an artificial intelligence feature into existing PSA solutions and how different user interface patterns are used to achieve a positive user experience by creating an artifact and surveying the users.

Artificial intelligence has become a new daily part of applications, and it is necessary to analyse these new tools and how to integrate them into existing solutions. The usability of artificial intelligence features differs a lot from traditional user interfaces. This is why these kinds of studies are needed because the emerging new technologies have not been studied before in the PSA context.

The artificial intelligence agent feature was successfully developed and integrated into the case application. It was mostly well received amongst the survey participants. The SUS rating was average, while the open-ended notes mentioned the positive modern look and UX of the system as well as the work hour actions. While the flaws that the participants commented on were about the slowness and about the AI agent answering wrong and hallucinating.

The survey results suggest that animated response and thinking components are well liked by users and should be used when implementing an AI agent tool. The logic of the AI agent is also important for the user experience and should be improved in the future before the release of this AI agent feature. The response times, especially when agent is creating suggestions, must be improved or indicated better to the user so that they understand the current state of the system.

The result of this thesis suggests future research to be done about chatbots in PSA solutions especially with the surveys made specifically for AI tools. This stems from the limitation that this thesis only analysed the user interface of the AI agent and not the logic behind it. This would benefit the future AI agent development to better understand how the logic affects the user experience with these kinds of AI agents.

# References


[1] "The 2025 AI Index Report | Stanford HAI." Accessed: Oct. 30, 2025. [Online]. Available: https://hai.stanford.edu/ai-index/2025-ai-index-report

[2] G. E. Anani, E. Nyamekye, and D. Bafour-Koduah, "Using artificial intelligence for academic writing in higher education: the perspectives of university students in Ghana," *Discov. Educ.*, vol. 4, no. 1, p. 46, Feb. 2025, doi: 10.1007/s44217-025-00434-5.

[3] F. Ricca, B. García, M. Nass, and M. Harman, "Next-Generation Software Testing: AI-Powered Test Automation," *IEEE Softw.*, vol. 42, no. 4, pp. 25–33, Jul. 2025, doi: 10.1109/MS.2025.3559194.

[4] A. Abdellatif, K. Badran, D. E. Costa, and E. Shihab, "A Comparison of Natural Language Understanding Platforms for Chatbots in Software Engineering," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3087–3102, Aug. 2022, doi: 10.1109/TSE.2021.3078384.

[5] "What is Agentic AI? | IBM." Accessed: Jan. 23, 2026. [Online]. Available: https://www.ibm.com/think/topics/agentic-ai

[6] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey," *IEEE Access*, vol. 13, pp. 18912–18936, 2025, doi: 10.1109/ACCESS.2025.3532853.

[7] N. Pellas, "AI Chatbots in Philology: A User Experience Case Study of Conversational Interfaces for Content Creation and Instruction," *Multimodal Technol. Interact.*, vol. 9, no. 7, pp. 65–100, 2025, doi: 10.3390/mti9070065.

[8] "The System Usability Scale - Rebecca A. Grier, Aaron Bangor, Philip Kortum, S. Camille Peres, 2013." Accessed: Mar. 10, 2026. [Online]. Available: https://journals.sagepub.com/doi/10.1177/1541931213571042

[9] "Validation of the System Usability Scale (SUS)." Accessed: Jan. 25, 2026. [Online]. Available: https://journals.sagepub.com/doi/epdf/10.1177/1541931213571043

[10] "Re-examining the chatBot Usability Scale (BUS-11) to assess user experience with customer relationship management chatbots," *Pers. Ubiquitous Comput.*, vol. 28, no. 6, pp. 1033–1044, Dec. 2024, doi: 10.1007/s00779-024-01834-4.

[11] S. Holmes, R. Bond, A. Moorhead, J. Zheng, V. Coates, and M. McTear, "Towards Validating a Chatbot Usability Scale," in *Design, User Experience, and Usability*, A. Marcus, E. Rosenzweig, and M. M. Soares, Eds., Cham: Springer Nature Switzerland, 2023, pp. 321–339. doi: 10.1007/978-3-031-35708-4_24.

[12] "Human-Computer Interaction | Research Starters | EBSCO Research," EBSCO. Accessed: Jan. 24, 2026. [Online]. Available: https://www.ebsco.com/research-starters/computer-science/human-computer-interaction

[13] J. Kaye, J. Teevan, V. Bellotti, and L. Wilcox, "HCI and AI in Industry: Current and Future," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, in CHI EA '24. New York, NY, USA: Association for Computing Machinery, May 2024, pp. 1–5. doi: 10.1145/3613905.3643974.

[14] J. P. Grodniewicz and M. Hohol, "Therapeutic Chatbots as Cognitive-Affective Artifacts," *Topoi*, vol. 43, no. 3, pp. 795–807, Aug. 2024, doi: 10.1007/s11245-024-10018-x.

[15] Q. N. Nguyen, A. Sidorova, and R. Torres, "User interactions with chatbot interfaces vs. Menu-based interfaces: An empirical study," *Comput. Hum. Behav.*, vol. 128, p. 107093, Mar. 2022, doi: 10.1016/j.chb.2021.107093.

[16] W. Huang, K. F. Hew, and L. K. Fryer, "Chatbots for language learning—Are they really useful? A systematic review of chatbot-supported language learning," *J. Comput. Assist. Learn.*, vol. 38, no. 1, pp. 237–257, 2022, doi: 10.1111/jcal.12610.

[17] U. Gnewuch, S. Morana, M. T. P. Adam, and A. Maedche, "Opposing Effects of Response Time in Human–Chatbot Interaction," *Bus. Inf. Syst. Eng.*, vol. 64, no. 6, pp. 773–791, Dec. 2022, doi: 10.1007/s12599-022-00755-x.

[18] J. H. Lee, H. Yang, D. Shin, and H. Kim, "Chatbots," *ELT J.*, vol. 74, no. 3, pp. 338–344, Aug. 2020, doi: 10.1093/elt/ccaa035.

[19] A. Srinivasaprasad, K. Sepanloo, S. N. Jahromi, D. Yu, and V. G. Duffy, "How AI Chatbot Response Style Affects Cognitive Load and Performance in Educational Tasks," in *HCI International 2025 – Late Breaking Papers*, V. G. Duffy, Ed., Cham: Springer Nature Switzerland, 2026, pp. 188–200. doi: 10.1007/978-3-032-13012-9_14.

[20] A. P. Chaves and M. A. Gerosa, "How Should My Chatbot Interact? A Survey on Social Characteristics in Human–Chatbot Interaction Design.," *Int. J. Hum.-Comput. Interact.*, vol. 37, no. 8, pp. 729–758, Apr. 2021, doi: 10.1080/10447318.2020.1841438.

[21] A. Boduch, *Flux Architecture : Design and Build Modern JavaScript Web Applications*. in Community Experience Distilled. Birmingham, UK: Packt Publishing, 2016. Accessed: Jan. 31, 2026. [Online]. Available: https://research.ebsco.com/linkprocessor/plink?id=4f16f4a9-ecca-34da-bfdf-5960f535ccc5

[22] S. Rathinam, "Analysis and Comparison of Different Frontend Frameworks," in *Applications and Techniques in Information Security*, S. Prabhu, S. R. Pokhrel, and G. Li, Eds., Singapore: Springer Nature, 2023, pp. 243–257. doi: 10.1007/978-981-99-2264-2_19.

[23]"Technology | 2025 Stack Overflow Developer Survey." Accessed: Feb. 01, 2026. [Online]. Available: https://survey.stackoverflow.co/2025/technology#1-web-frameworks-and-technologies

[24]"react," npm. Accessed: Feb. 01, 2026. [Online]. Available: https://www.npmjs.com/package/react

[25]F. Ferreira, H. S. Borges, and M. T. Valente, "On the (un-)adoption of JavaScript front-end frameworks," *Softw. Pract. Exp.*, vol. 52, no. 4, pp. 947–966, 2022, doi: 10.1002/spe.3044.

[26]F. Ferreira, H. S. Borges, and M. T. Valente, "Refactoring react-based Web apps," *J. Syst. Softw.*, vol. 215, p. 112105, Sep. 2024, doi: 10.1016/j.jss.2024.112105.

[27]"Understanding Your UI as a Tree – React." Accessed: Feb. 06, 2026. [Online]. Available: https://react.dev/learn/understanding-your-ui-as-a-tree

[28]"Built-in React Hooks – React." Accessed: Feb. 06, 2026. [Online]. Available: https://react.dev/reference/react/hooks

[29]A. Morchid *et al.*, "IoT-enabled smart agriculture for improving water management: A smart irrigation control using embedded systems and Server-Sent Events," *Sci. Afr.*, vol. 27, p. e02527, Mar. 2025, doi: 10.1016/j.sciaf.2024.e02527.

[30]S. Vinoski, "Server-Sent Events with Yaws," *IEEE Internet Comput.*, vol. 16, no. 5, pp. 98–102, Sep. 2012, doi: 10.1109/MIC.2012.117.

[31]"Server-Sent Events: The Perfect Match for Real-Time Chat." Accessed: Feb. 06, 2026. [Online]. Available: https://community.ibm.com/community/user/blogs/anjana-m-r/2025/10/03/server-sent-events-the-perfect-match-for-real-time

[32]A. Hevner *et al.*, "Design Science in Information Systems Research," *Manag. Inf. Syst. Q.*, vol. 28, p. 75, Mar. 2004.

[33]R. J. Wieringa, "What Is Design Science?," in *Design Science Methodology for Information Systems and Software Engineering*, R. J. Wieringa, Ed., Berlin, Heidelberg: Springer, 2014, pp. 3–11. doi: 10.1007/978-3-662-43839-8_1.

[34]K. PEFFERS, T. TUUNANEN, M. A. ROTHENBERGER, and S. CHATTERJEE, "A Design Science Research Methodology for Information Systems Research.," *J. Manag. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/MIS0742-1222240302.

[35]"The WebSocket API (WebSockets) - Web APIs | MDN," MDN Web Docs. Accessed: Mar. 07, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

[36]"XMLHttpRequest - Web APIs | MDN," MDN Web Docs. Accessed: Mar. 07, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest

[37] "Using server-sent events - Web APIs | MDN," MDN Web Docs. Accessed: Nov. 15, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

[38] R. Mahdavi-Hezaveh, N. Ajmeri, and L. Williams, "Feature toggles as code: Heuristics and metrics for structuring feature toggles," *Inf. Softw. Technol.*, vol. 145, p. 106813, May 2022, doi: 10.1016/j.infsof.2021.106813.

[39] M. K. Shen and D. Yoon, "The Dark Addiction Patterns of Current AI Chatbot Interfaces," in *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, in CHI EA '25. New York, NY, USA: Association for Computing Machinery, huhtikuu 2025, pp. 1–7. doi: 10.1145/3706599.3720003.

[40] T.-S. Kim, M. John Ignacio, S. Yu, H. Jin, and Y.-G. Kim, "UI/UX for Generative AI: Taxonomy, Trend, and Challenge," *IEEE Access*, vol. 12, pp. 179891–179911, 2024, doi: 10.1109/ACCESS.2024.3502628.

[41] "Art. 17 GDPR – Right to erasure ('right to be forgotten')," General Data Protection Regulation (GDPR). Accessed: Mar. 28, 2026. [Online]. Available: https://gdpr-info.eu/art-17-gdpr/

[42] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2024. doi: 10.1007/978-3-662-69306-3.

[43] A. Fink, *How to Sample in Surveys*. SAGE Publications, Inc., 2003. doi: 10.4135/9781412984478.

[44] J. R. Lewis, "The System Usability Scale: Past, Present, and Future.," *Int. J. Hum.-Comput. Interact.*, vol. 34, no. 7, pp. 577–590, Jul. 2018, doi: 10.1080/10447318.2018.1455307.

Appendix 1. Summary of the survey results

| Section | Question | Average | Median |
|---|---|---|---|
| AI agent specific questions | Adding work hours from the work hour -view is a time-consuming task. | 3 | 3 |
| | Adding work hours from the new AI agent is a time-consuming task compared to the existing work hour -view. | 2.71 | 3 |
| | The AI agent's thinking element helps me to understand what it is doing. | 4.29 | 4 |
| | The animated response allows me to read the response before it is complete. | 4.43 | 5 |
| System Usability Scale (SUS) | I think that I would like to use AI agent frequently | 3.43 | 3 |
| | I found the AI agent unnecessary complex. | 2.29 | 2 |
| | I thought the AI agent was easy to use. | 3.86 | 4 |
| | I think that I would need the support of a technical person to be able to use AI agent. | 1.71 | 1 |
| | I found the AI agent functions were well integrated into case application. | 3.14 | 3 |
| | I thought there was too much inconsistency in AI agent. | 2.86 | 3 |
| | I would imagine that most people would learn to use AI agent very quickly. | 4.00 | 4 |
| | I found AI agent very cumbersome to use. | 2.29 | 2 |
| | I felt very confident using AI agent. | 3.43 | 4 |
| | I needed to learn a lot of things before I could get going with AI agent. | 1.43 | 1 |

| | | | |
|---|---|---|---|
| Background | | Experience with case application | Basic tasks: 5; Complex tasks: 1; Most features: 1 |
| | | Familiarity with AI tools (ChatGPT, Gemini, etc.) | Daily: 6; Occasionally: 1 |
| | | How often do you use case application? | Daily: 4; Weekly: 2; Monthly: 1 |
| Open ended | What did you like most about AI agent? | Familiar UX patterns, easy add work hours button;<br><br>Easy-to-use, modern look, and for the most parts very intuitive;<br><br>Hour entry suggestions;<br><br>Data retrieval and answering based on data seems pretty useful;<br><br>multiple entries at the same time; | |
| | What was the most frustrating part of using AI agent? | Hallucination/inconsistency/inaccuracy which makes it very inefficient to use.;<br><br>I asked it to enter eight hours of project management work for a project for every work day in the past week. It suggested 7.5hrs for Monday, Tuesday, and Thursday (which given my work hour history is accurate but I specifically asked for 8hrs, and still got zero 8hr suggestions).<br>It didn't suggest anything for Friday which was also a regular work day. And, for whatever reason, it suggested 2 hours of wrong work type for Wednesday, and 5hrs of project management.<br>Furthermore, I asked it to add the work hours to a project called "Super cool project". The AI responded with: | |

| | | |
|---|---|---|
| | | "I've suggested the following work hour entries for "Project management" on the phase "Super cool project" (project: Super cool project) for every workday last week"<br>But all of the work hour suggestions were for a completely different project (with a very different name), and even added the hours to a seemingly random phase called "C". The work hour suggestions do not tell anything about the specific phase to which the hours are entered either.<br>On a side note, the conversation history drawer did not feel very intuitive to me. I noticed it only had one clearly interactive item for me (since I don't have any earlier conversations) "User Work Hours Query for Last Week". I was not sure what that even is, and clicking it simply closed the drawer with seemingly no effect. I understand now how it is supposed to work but perhaps if the user only has one conversation, and the user is already on that page, it should have some kind of an empty state or something like this.;<br><br>Once the AI confirms it's understood your work hour entry request, there's roughly a 5 second wait before the suggested entries actually appear. At first, it felt like something was off. Maybe the status could stand out a bit more in this particular case?;<br><br>That when you are entering hours with AI you are not on WTE page, have to keep to tabs open and jump between them..;<br><br>Slowness on suggestions and invalid suggestions;<br><br>some hallucinations; |